Image_Segmentation

June 14, 2025

Dela Cruz, Cloie Sofia - Morales, Rayleen Claire / AM1

IMPORT LIBRARIES

[1]:
```
#This import is OpenCV (Open Source Computer Vision) Library, which is used for␣
 ↪computer vision, machine learning and image processing (Kulhary, 2019).
#For this activity, this import is mainly used for image processing which has␣
 ↪helped with loading the image, perform image techniques such as masking and␣
 ↪thresholding.
import cv2
```

[2]:
```
#NumPy, being short for Numerical Python, is the common library for numbers and␣
 ↪arrays to assist in faster processing (W3Schools, 2022).
#In this activity, this has been used for arrays intended for hsv value␣
 ↪storing, kernel shape storing, and even area computing for storing later.

import numpy as np
```

[3]:
```
#Pyplot assists in plotting and displaying images, which help provide a simple␣
 ↪yet interactive plot generation to custom image display to the user's wants␣
 ↪(Matplotlib, 2025).
#This library has been used to set figure sizes, add titles to images, and␣
 ↪embed display on the notebook itself.
import matplotlib.pyplot as plt
```

[4]:
```
#This module provides access to parameters and functions relating to the system␣
 ↪and interaction with the interpreter (Python, 2010).
#This is import is to execute the instructions of the activity that exits the␣
 ↪program if the loading is not successful.

import sys
```

LOADING THE IMAGE

[5]:
```
#The image for this activity was added to the environment and the folder.
#The path was stored in a variable for easier access and modification if image␣
 ↪were to change paths or to change the image itself.
```

```
img_path = 'genshin.png'
```

[6]:
```
#This utilizes the cv2 function 'imread' to load the image.
#This uses the path variable to load and store the image in 'img'.

img = cv2.imread(img_path)
```

[7]:
```
#This whole if function is to follow the provided instructions in the activity guide to exit the program once image loading fails.
#This basically checks if 'img' has its stored contents, if not, it prints the error and the provided path in the program then follows the instructed exit once loading fails.

if img is None:
    print(f"Load Error: Could not load image from {img_path}")
    sys.exit(1)
```

[8]:
```
#Apparently, OpenCV has BGR has its default color space due to camera manufacturers and software libraries doing the same (RoboFlow, 2025).
#This just basically converts the default BGR color of the image to the common RGB, which is essential for image displaying later.

rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

DISPLAYING THE IMAGE

Below will be the commonly used functions to display the image. To avoid redundancy of sentences, line per line explanation of the methods will be explained in the cell below, only small changes will be specified per line later.

[9]:
```
#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in inches.
plt.figure(figsize=(12, 6))

#This function specifies which stored image will be displayed, in this case, it is the RGB Image.
#The variables inputted will be rendered as a pseudocolor image (Matplotlib, 2025b).
plt.imshow(rgb_img)

#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer which presents the next splashscreen for the game.
plt.title('Genshin Impact 5.7 Update')

#Since pyplot mainly works with plot generation, normally axes will be seen.
```

```
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')

#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



IMAGE PREPROCESSING

[10]:
```
#This function basically converts the img to grayscale and stores in the␣
 ↪'gray_img' variable.
#Take note that the 'img' was used here which was BGR, hence BGR2GRAY was used,␣
 ↪not RGB2GRAY.
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Gray Image.
```

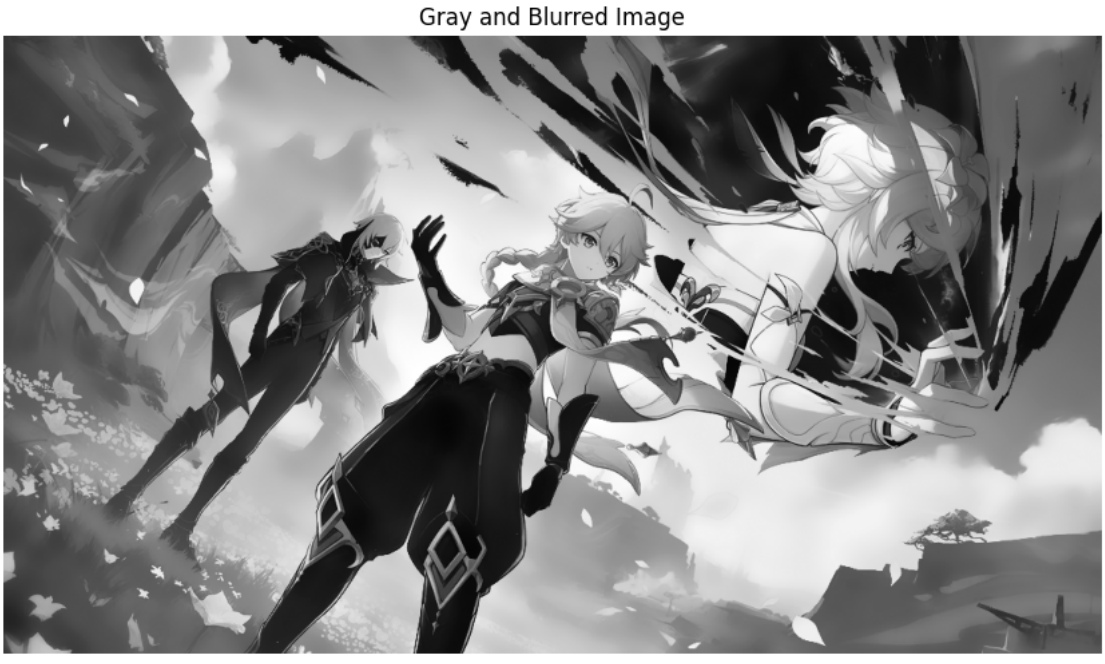
```
#Additionally, cmap is an extra parameter for grayscale images, without this␣
 ↪the image would appear yellowish rather than grayscale.
plt.imshow(gray_img, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Gray Image')
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



[11]:
```
#Apparently, blurring is helpful in noise removal, image smoothing, and removal␣
 ↪of low intensity edges to focus on the main interest in the image␣
 ↪(GeeksforGeeks, 2019).
#With that in mind, bilateral blur was a better choice than gaussian as per our␣
 ↪trial and error as it gave emphasis to the three persons in the image while␣
 ↪smoothing the background.
#Parameters of the blur included diameter for the pixel size, sigmaColor for␣
 ↪the value for mixing colors, and sigmaSpace for the value for mixing nearby␣
 ↪pixels (GeeksforGeeks, 2019b).
```

```
#This basic information of the purpose of the parameters, along with the goal␣
 ↪of bringing focus to the three figures, has helped in the trial and error of␣
 ↪choosing the values for the function parameters.
bltl = cv2.bilateralFilter(gray_img, 25, 25, 75)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Bilateral Blur Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(bltl, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Gray and Blurred Image')
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```

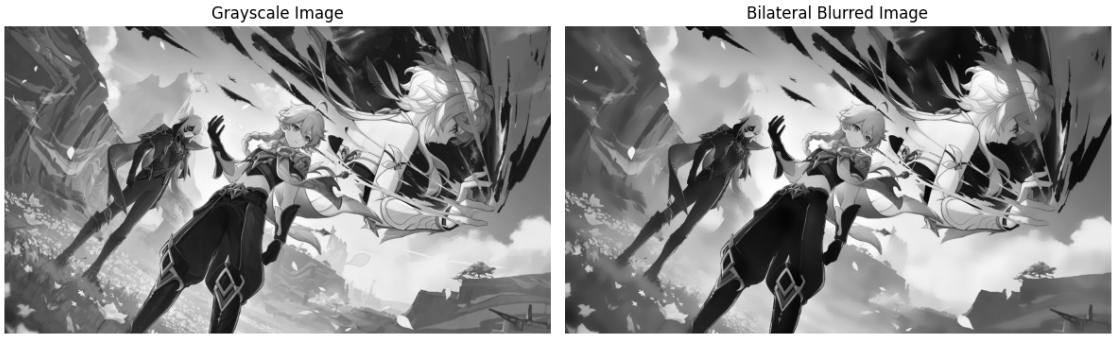


[12]:
```
#This line of code helps plot the images side by side, as per the instruction␣
 ↪in the activity guide.
#The 'subplot' function is intended to make subplots in one call, wherein it is␣
 ↪stored in the 'fig' object that holds the entire plot.
#'Axes' similarly holds the two figures together with 1 row, 2 columns, and a␣
 ↪figure size of 12 inches width and 6 inches height for each image.
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

#This helps setup the grayscale image, 'gray_img' for displaying.
#Cmap is still present as this is a grayscale image.
axes[0].imshow(gray_img, cmap='gray')
#This sets the title of the first image plotted and displayed to be "Grayscale␣
 ↪Image".
#Axes[0] refers to the first figure stored in the figure array of size two.
axes[0].set_title('Grayscale Image')
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the first image.
#The removal of the axis on the first image is to be able to clearly compare␣
 ↪the differences with the second image.
axes[0].axis('off')

#This helps setup the grayscale and blurred image, 'bltl' for displaying.
#Cmap is still present as this is a grayscale image.
axes[1].imshow(bltl, cmap='gray')
#This sets the title of the first image plotted and displayed to be "Bilateral␣
 ↪Blurred Image".
#Axes[1] refers to the second figure stored in the figure array of size two.
axes[1].set_title('Bilateral Blurred Image')
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the first image.
#The removal of the axis on the second image is also to be able to clearly␣
 ↪compare the differences with the first image.
axes[1].axis('off')

#This function helps in the padding between the subplots (Matplotlib, 2025c).
#With no parameters present in the code below, the default parameters were used.
plt.tight_layout()
#This displays the entire display that has been plotted using the previous␣
 ↪commands.
#Observing the image, you can see that the three figures are still very␣
 ↪prominent, while the background has a blurry and smoothened finish.
plt.show()
```



SEGMENTATION TECHNIQUES

TECHNIQUE A: Automatic Thresholding (Otsu's Method)

[13]:
```
#Hence its name, this technique determines the value of the threshold␣
 ↪automatically (GeeksforGeeks, 2019b).
#Thresholding techniques invoved are thresh_binary so only two values will be␣
 ↪used and thresh_ostu for the automatic thresholding.
ret, otsu_thresh = cv2.threshold(bltl, 150, 255, cv2.THRESH_BINARY + cv2.
 ↪THRESH_OTSU)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Otsu Thresholding Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(otsu_thresh, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Otsu Thresholding')
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()

#This prints the automatic threshold value calculated by Otsu's Method.
```

```
#This is needed because the 150 in the threshold function was disregarded due␣
 ↪to the THRESH_OTSU function.
print("Calculated Threshold: ", ret)
```




```
Calculated Threshold:  136.0
```

[14]:
```
#This is the Binary Inverse Technique that has been told to observe for the␣
 ↪research question.
#This reverses the binary values based on the threshold value, 150.
inv_ret, inv_thresh = cv2.threshold(bltl, 150, 255, cv2.THRESH_BINARY_INV)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Binary Inverse Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(inv_thresh, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Binary Inverse Thresholding')
#Since pyplot mainly works with plot generation, normally axes will be seen.
```

```
#This function turns of the display of the axes, mainly to focus on the image displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting commands done earlier.
plt.show()
```



1) What is the ret value in the cv2.threshold output, and what does THRESH_OTSU do?

The ret value in the threshold output is the calculated threshold of the THRESH_OTSU, which was printed to be 136.0. The thresholding technique function THRESH_OTSU helps automatically calculate the best threshold value using an image histogram (Murzova & Seth, 2020). This also means that the 150 value in the threshold value parameters is disregarded.

2) How might using THRESH_BINARY_INV change the output?

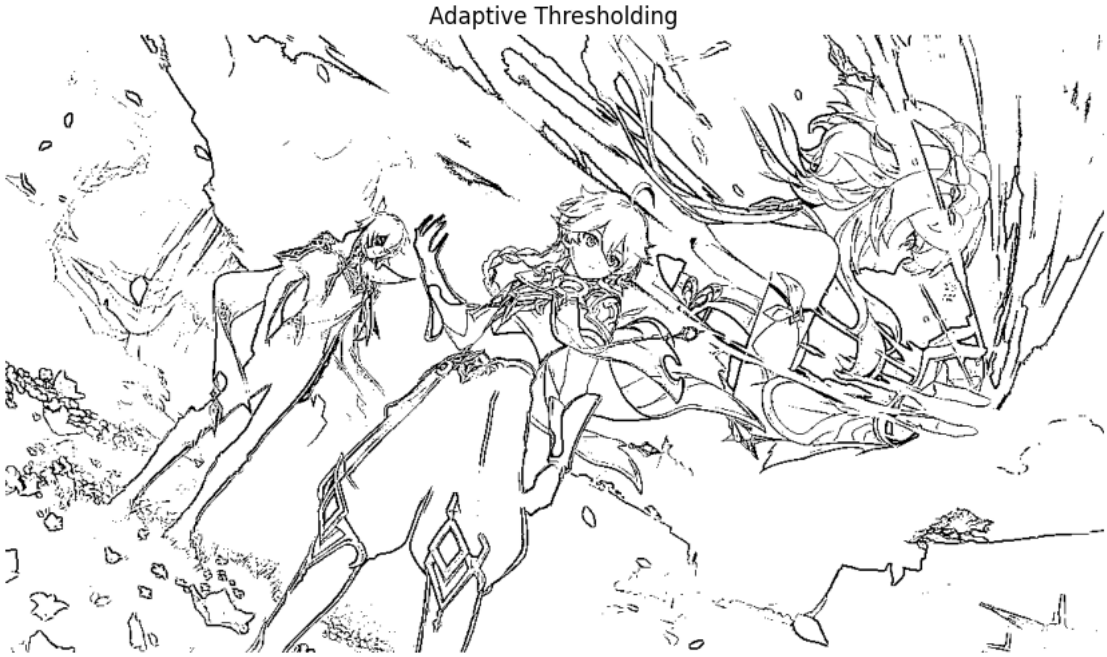
THRESH_BINARY_INV utilizes the threshold value of 150 and inverses the binary value. Anything below 150 becomes white, as 255 is the max value, while anything above 150 becomes black.

TECHNIQUE B: Adaptive Thresholding:

[15]:
```
#This method uses adaptive thresholding, wherein the threshold value is calculated, giving different thresholds for each region (TutorialsPoint, 2025).
#The adaptive method used here is the 'ADAPTIVE_THRESH_MEAN_C' that gets the mean of the neighborhood area.
```

```
#Based on the other parameters, the block size for the neighborhood is 5, while␣
 ↪the constant used for calculation is 9.
#Once again, the resulting image was done with a trial and error of the␣
 ↪parameters
adapt_thresh = cv2.adaptiveThreshold (bltl, 255, cv2.ADAPTIVE_THRESH_MEAN_C,␣
 ↪cv2.THRESH_BINARY, 5, 9)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Adaptive Thresholding Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(adapt_thresh, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Adaptive Thresholding')
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



1) When would you use adaptive thresholding instead of simple (Otsu's) thresholding?

Adaptive Thresholding takes into consideration the neighbor pixels, thus this is better for images with non-uniform lighting conditions and complex intensity distributions (Buhl, 2023). A simple thresholding is best suited for uniform images that creates clear bimodal histograms. The images used in this activity has varying light and color intensities, thus adaptive thresholding is better for this.

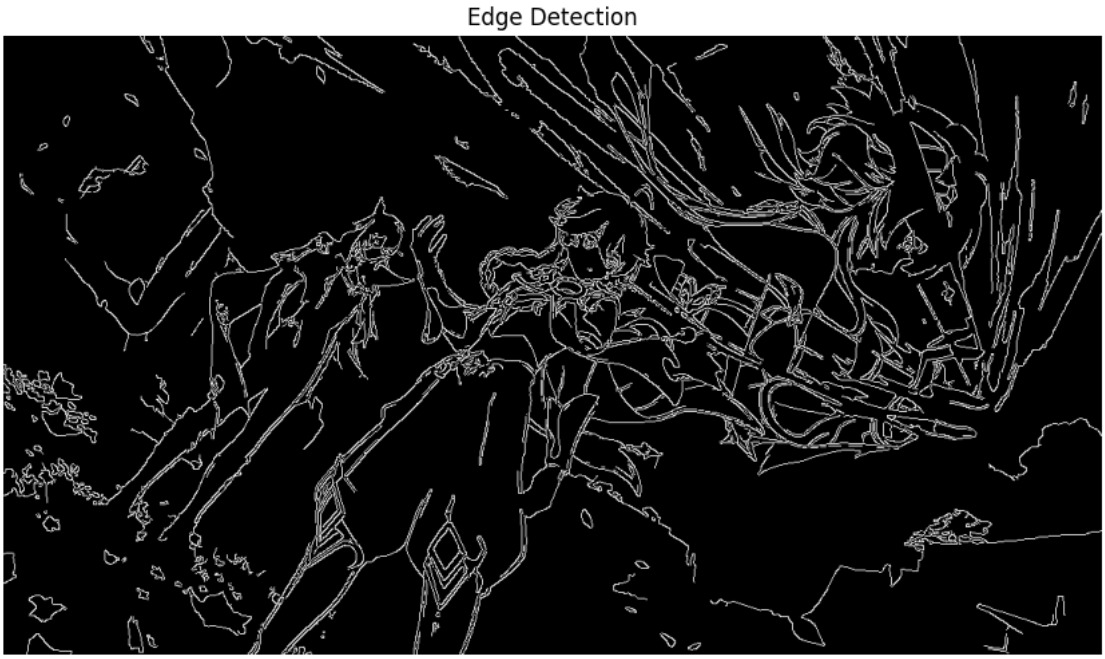
2) Explain the roles of blockSize and C in adaptiveThreshold.

Block Size in the parameters helps define the size of the neighborhood area, in the case above, it is 5. This creates a 5x5 window where the threshold is actually calculated. On the otherhand, the C parameter is a constant that is subtracted from the mean calculated (TutorialsPoint, 2025). This is meant for fine-tuning the threshold.

TECHNIQUE C: Edge Detection (Canny)

[16]:
```
#Canny Edge Detection is utilized here for noise reduction and edge thinning␣
 ↪(GeeksforGeeks, 2016).
#The parameters of the function below uses 100 as the lower threshold and 300␣
 ↪for the upper threshold.
#Any pixels with gradient magnitude lower than 100 are not considered edges,␣
 ↪anything in between 100 and 300 are weak edges, and anything above 300 are␣
 ↪strong edges.
canny = cv2.Canny(bltl, 100, 300)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Canny Edge Detection Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(canny, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Edge Detection')
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



1) Describe the multi-stage process of the Canny edge detector.

The Canny Edge Detection undergoes multiple processes of (1)noise reduction, (2) intensity gradient identification, (3) non-maximum supression, and (4) hysteresis thresholding (GeeksforGeeks, 2016).

First, noise reduction is done by blurring the image through a Gaussian filter. Second process involves finding the intensity gradient of the image using a Sobel operator. This helps identify both the horizontal and vertical gradients to attain the magnitude of the edges and its direction. Third process is the non-maximum suppression that preserves the sharpest edges through thinning them down. This is done by comparing the pixel with its neighbors and its gradient direction, after which the decision whether it is suppressed or kept. The final process in the Canny Algorithm is the Double Thresholding to filter the weak, strong, and non-edges to reveal prominent details and edges, while keeping weak edges that contribute to the strong ones.

2) What is the significance of the two threshold values (threshold1 and threshold2)?

The significance of the two threshold values are for the final stage of the edge detection process. It has been mentioned above that weak, strong, and non-edges are considered. Thus the threshold1 (lower threshold) sets a filter or a boundary to which pixels are not considered edges (below threshold1) and weak edges (above threshold1 but below threshold2). And the threshold2 (upper threshold) sets a boundary to which pixels will be considered as weak edges (above threshold1 but below threshold2) and strong edges (above threshold2).

TECHNIQUE D: Color Filtering (HSV)

[17]:
```
#Color Filtering utilizes a given HSV value of a color to separate the regions␣
↪of an image (GeeksforGeeks, 2019c).
#This line of code converts the rgb_img with the RGB color space to an HSV one.
```

```
hsv_img = cv2.cvtColor(rgb_img, cv2.COLOR_RGB2HSV)

#This block of code focuses mainly on the darker parts of the image (mostly␣
 ↪clothes).
#Below, numpy is used to specify and store the HSV values of the colors in the␣
 ↪image in an array.
#It is observable that this block of code has a lower and upper range, this␣
 ↪captures the color along the given values.
#This HSV color range was actually a mix of trial and error along with the␣
 ↪searching of the appropriate HSV values for the color desired to capture.
#Lots of color ranges were tried here along with red, orange, and even green,␣
 ↪however only utilizing the two colors provided in this code has preserved␣
 ↪most of the significant part of the picture and disregards the background.
lower_dark = np.array([0, 0, 0])
upper_dark = np.array([180, 100, 70])

#This block of code focuses mainly on the lighter parts of the image (mostly␣
 ↪skin).
#Below, numpy is used to specify and store the HSV values of the colors in the␣
 ↪image in an array.
#It is observable that this block of code has a lower and upper range, this␣
 ↪captures the color along the given values.
#The color range for this actually captures majority of the image, even the␣
 ↪background, as the skin of the 2 figures in the image is in the same range␣
 ↪as the background.
#Furthermore, the flowers below and details in the mountains were also captured␣
 ↪due to a similar color range.
lower_light = np.array([0, 20, 80])
upper_light = np.array([25, 150, 255])

#These two lines of code does the same thing: masking.
#Despite similar purposes, the first line of this block masks the darker parts␣
 ↪of the image, which mostly captures the clothes of the two figures in the␣
 ↪image.
#The second line masks the lighter parts of the image including the hair, the␣
 ↪flowers, and the orange-y sky background.
#The function 'inRange' helps in thresholding the image based on color ranges␣
 ↪provided (upper and lower), creating a mask that falls within the captured␣
 ↪color regions.
mask_dark = cv2.inRange(hsv_img, lower_dark, upper_dark)
mask_light = cv2.inRange(hsv_img, lower_light, upper_light)

#This line of code combines the two masks together, the lighter mask and the␣
 ↪darker mask.
#The 'bitwise_or' helps perform the bitwise OR operation the two masks␣
 ↪together, joining even their differences.
```

```
combined_mask = cv2.bitwise_or(mask_light, mask_dark)

#This line of code performs the bitwse AND operation to the inputs, as both of␣
 ↪the inputs are both rgb_img, it returns the same image.
#However, the mask parameters provides a mask to the original 'rgb_img', thus␣
 ↪printing an RGB Image of the white spaces in the mask.
highlighted_img = cv2.bitwise_and(rgb_img, rgb_img, mask=combined_mask)
```

[18]:
```
#Once again, this sets a plot to be called with a width of 12 inches and a␣
 ↪height of 12 inches.
#This will be applied to the two images below.
plt.figure(figsize=(12, 12))

#This block of code focuses on the first image, which is the combined mask.
#The parameters under the subplot provide 2 rows, 1 column, and an index of 1␣
 ↪for the code.
plt.subplot(2,1,1)
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the first image.
#The removal of the axis on the first image is also to be able to clearly␣
 ↪compare the differences with the second image.
plt.axis('off')
#This helps setup the binary colored image, 'combined_mask' for displaying.
#Cmap is still present as this is a black and white image.
plt.imshow(combined_mask, cmap='gray')
#This sets the title of the first image plotted and displayed to be "Combined␣
 ↪Mask".
#This image displays the combined masks of the lighter color range and the␣
 ↪darker color range.
plt.title('Combined Mask')

#This block of code focuses on the second image, which is the Color Filtered␣
 ↪Image.
#The parameters under the subplot provide 2 rows, 1 column, and an index of 2␣
 ↪for the code.
plt.subplot(2,1,2)
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the first image.
#The removal of the axis on the second image is also to be able to clearly␣
 ↪compare the differences with the first image.
plt.axis('off')
#This helps setup the colored RGB image, 'highlighted_img' for displaying.
#Cmap is not present as it is not a grayscale image.
plt.imshow(highlighted_img)
#This sets the title of the second image plotted and displayed to be "Color␣
 ↪Filtering".
```

```
#This image displays the colored masked image.
plt.title('Color Filtering')

#This function helps in the padding between the subplots (Matplotlib, 2025c).
#With no parameters present in the code below, the default parameters were used.
plt.tight_layout()

#This displays the entire display that has been plotted using the previous␣
 ↪commands.
#Observing the image, you can see that the mask is not clean as this image is␣
 ↪very complex with its colors.
#You can even notice the missing parts of the clothes and the skin.
#While this was adjusted during the trial and error of the color ranges, if␣
 ↪more were added, the entire picture would not be masked.
#Filtering out with more HSV color ranges and being familiar with the HSV␣
 ↪values is a crucial skill in using the color filtering technique.
plt.show()
```



1) Why is HSV color space often preferred over RGB for color-based image segmentation?

The HSV color space actually simplifies the segmentation, making it useful for object detection, color segmentation, and analytical image processing (Jarvis, 2024). The simplification happens with the separation of color (hue), intensity (saturation), and brightness (value). The RGB color space is a mix of the red, blue, and green, making it difficult for image processing that has many light variants. HSV is useful with this as the hue maintains its color range, providing consistency and easier desire color capturing.

2) How would you programmatically find the HSV values for a specific color in your image (e.g., if you click on a pixel)?

Programmatically speaking, the OpenCV will still be utilized for the BGR to HSV color space conversion. You would have to have the original image and its HSV image (converted). If a pixel is specified, then its x and y can get the HSV value at that pixel. For clicking, a mouse event handler can be defined to catch mouse click events on the BGR image and get the location of the clicked pixel to return the HSV value using the same coordinates for the x and y. Take note that this utilizes the cvtColor function and the BGR2HSV function, and the HSV image will take [y, x] values. Basically, mouse clicks are registered then the coordinates will be used to access the HSV value on the converted image.

MORPHOLOGICAL OPERATIONS

[19]:
```
#This line of code follows the instruction of creating a kernel of given shape␣
 ↪and type with ones (GeeksforGeeks, 2017).
#This is essential for the morphological operations to define the neighborhood␣
 ↪that a pixel will consider, in this case, it is a 3x3 kernel.
#This would also define the reach of the performed operations in the picture.
kernel = np.ones((3,3), np.uint8)
```

[20]:
```
#Erosion, on the other hand, is used for noise removal of white noises which␣
 ↪detaches white spaces.
#As observed, it increases the black spaces of the image and thins the white␣
 ↪edges or the object area.
#The erode function below also utilizes the set kernel in one iteration, but␣
 ↪this time processes the dilated image from above.
img_erode = cv2.erode(combined_mask, kernel, iterations=1)
```

[21]:
```
#As observed, the dilate function has increased white spaces, or the object␣
 ↪area, because of this this operation is also useful in joining broken parts␣
 ↪or edges in an object (GeeksforGeeks, 2017a).
#The dilate function utilizes the kernel, in this case 3x3, and dilates the␣
 ↪combined_mask (previously used in color filtering) for one iteration.
img_dilate = cv2.dilate(img_erode, kernel, iterations=1)
```

[22]:
```
#Once again, this sets a plot to be called with a width of 12 inches and a␣
 ↪height of 22 inches.
#This will be applied to the three images below.
plt.figure(figsize=(12, 20))

#This block of code focuses on the first image, which is the combined mask.
#The parameters under the subplot provide 3 rows, 1 column, and an index of 1␣
 ↪for the code.
plt.subplot(3,1,1)
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the first image.
```

```
#The removal of the axis on the first image is also to be able to clearly␣
 ↪compare the differences with the other images.
plt.axis('off')
#This helps setup the binary colored image, 'combined_mask' for displaying.
#Cmap is still present as this is a black and white image.
plt.imshow(combined_mask, cmap='gray')
#This sets the title of the first image plotted and displayed to be "Combined␣
 ↪Mask".
#This title basically provideees the description of the image, providing context␣
 ↪to the displayed image.
plt.title('Combined Mask')

#This block of code focuses on the second image, which is the eroded image.
#The parameters under the subplot provide 3 rows, 1 column, and an index of 2␣
 ↪for the code.
plt.subplot(3,1,2)
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the second image.
#The removal of the axis on the second image is also to be able to clearly␣
 ↪compare the differences with the other images.
plt.axis('off')
#This helps setup the eroded image, 'img_erode' for displaying.
#Cmap is still present as this is a black and white image.
plt.imshow(img_erode, cmap='gray')
#This sets the title of the second image plotted and displayed to be "Eroded␣
 ↪Image".
#This title basically provides the description of the image, providing context␣
 ↪to the displayed image.
plt.title('Eroded Image')

#This block of code focuses on the third image, which is the dilated image.
#The parameters under the subplot provide 3 rows, 1 column, and an index of 3␣
 ↪for the code.
plt.subplot(3,1,3)
#Like the initial plotting commands above, this line of code turns off the axis␣
 ↪for the third image.
#The removal of the axis on the third image is also to be able to clearly␣
 ↪compare the differences with the other images.
plt.axis('off')
#This helps setup the dilated image, 'img_dilate' for displaying.
#Cmap is still present as this is a black and white image.
plt.imshow(img_dilate, cmap='gray')
#This sets the title of the third image plotted and displayed to be "Dilated␣
 ↪Image".
#This title basically provides the description of the image, providing context␣
 ↪to the displayed image.
```

```
plt.title('Dilated Image')

#This function helps in the padding between the subplots (Matplotlib, 2025c).
#With no parameters present in the code below, the default parameters were used.
plt.tight_layout()

#This displays the entire display that has been plotted using the previous␣
 ↪commands.
#The picture undergoing the erosion first minimized the object area or the␣
 ↪white space, then after dilation, the object area has been expanded.
plt.show()
```

Combined Mask



Eroded Image



Dilated Image



1) Explain the difference between erosion and dilation.

Erosion removes small white noises and even separates objects, while dilation increases the object area (white spaces) or connects objects. From observation, I would differentiate them as erosion adding more black spaces in the image and dilation adding more white spaces in the image. In a general perspective, these two help refine the image through noise removal and safer object ara expansion.

2) How would applying these operations in sequence (e.g., erosion then dilation, or dilation then erosion) affect the image?

The sequences in applying the operations display different results as each operation serve different purposes. An opening operation is intended to remove small white noises with no worry to large object areas as they are preserved. This means that erosion shrinks object area to lessen white noises then dilation increases the area with less white noise back to its original size. A closing operation's goal is different, which initially uses dilation to fill in the black areas within object areas, then smoothens the contour of the dilated spaces once erosion takes place. The sequences matter but this depends on the image on what it needs to result to a more refined image.

3) What are "opening" and "closing" operations?

Opening operation is erosion then dilation, while closing operation is dilation then erosion. Each operation has a different goal, thus making the image have varying operation to execute depending on its need. The opening operation is mainly for white noise removal, while the closing operation closes holes or gaps in object areas.

FINDING AND DESCRIBING SHAPES: BLOB DETECTION

[23]:
```
#This line of code sets up the blob detector parameters by creating an object␣
 ↪to hold the parameters.
#The variable 'params' will now hold the instance of the parameters.
params = cv2.SimpleBlobDetector_Params()

#This line enables the blob filtering by their area.
#This helps consider the minimum and maximum area as the area range specified␣
 ↪later.
params.filterByArea = True
#This sets the minimum value for the blob to be considered.
#Only the objects above the value of 50 will be processed.
params.minArea = 50
#This sets the maximum value for the blob to be considered.
#Only the objects below the value of 10000 will be processed
params.maxArea = 10000

#This line enables the blob filtering based on color.
#Blobs that are represented by a certain pixel that is specified below will be␣
 ↪the only ones considered.
params.filterByColor = True
```

```
#This sets a color that the detector will look for in a binary/grayscale image,␣
 ↪which is 255 (white).
#Only white pixels will be considered as blobs.
params.blobColor = 255

#This enables the filter based on the circularity of a blob.
#Blobs that have a reasonable degree of circularity will be considered.
params.filterByCircularity = True
#This sets the minimum circularity value that a blob must be to be detected.
#In this case, only blobs with 0.1 circularity and above will be considered.
params.minCircularity = 0.1

#This enables filtering on how convex a blob is.
#Blobs that do not have a significant inward curves or deep indentations will␣
 ↪be considered.
params.filterByConvexity = True
#This sets the minimum convexity that a blob must have.
#Only blobs that have 0.1 and above convex value will be considered as a blob.
params.minConvexity = 0.1

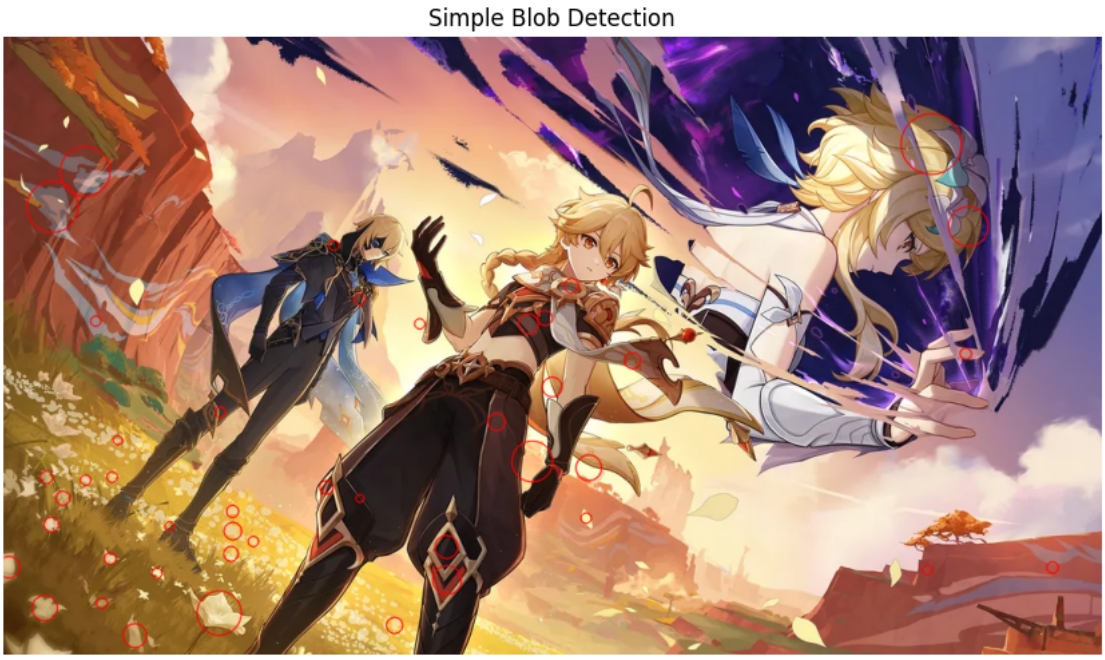
#The function below creates the instance of the SimpleBlobDetector using the␣
 ↪params variable.
#All the filters that have been set earlier are applied and is being held by␣
 ↪'detector'.
detector = cv2.SimpleBlobDetector_create(params)

#Using the 'combined_mask' binary image, it will detect blobs using the␣
 ↪variable instance of the blob detector, 'detector.'
#The detected blobs that align with the filters will be stored in the␣
 ↪'keypoints' list.
keypoints = detector.detect(combined_mask)

#This helps copy the original RGB image, then stored in 'blobs'
#This will be useful for printing the blobs along with the original image.
blobs = rgb_img.copy()
#This function helps draw the blobs in red circles, with the stored information␣
 ↪of its areas in the keypoints list.
#The flag 'DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS' will draw circles on the␣
 ↪keypoints with a size proportional to the property of keypoints.
blob_circles = cv2.drawKeypoints(blobs, keypoints, None, (255, 0, 0), cv2.
 ↪DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)


#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
```

```
plt.figure(figsize=(12, 6))
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Blob Detection Image.
#The variables inputted will be rendered as a pseudocolor image (Matplotlib,␣
 ↪2025b).
plt.imshow(blob_circles)
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Simple Blob Detection')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



[24]:
```
#This line of code just provides details for the output below.
#This will be a looping output, hence the need for a title and context for its␣
 ↪outputs.
print("Detected Blob Details:")

#This is a loop that displays the properties of the blobs drawn.
```

```
#The keypoints list is utilized here and accessed for each iteration to print its details.
for i, kp in enumerate(keypoints):
    #This gets the coordinates of the blob center as a float type.
    #Each item in the list will have its stored location for printing later.
    x, y = kp.pt              # Coordinates of the blob center (as float)
    #This gets the size or diameter of the blob in keypoints.
    #Each item in the list will have its stored size for printing later.
    size = kp.size
    #This just basically prints the number of the blob, its location thorough coordinates, and its size.
    #Basically, it prints what have been gathered above.
    print(f"Blob {i + 1}: Location = ({x:.2f}, {y:.2f}), Size = {size:.2f}")
```

```
Detected Blob Details:
Blob 1: Location = (387.03, 582.61), Size = 16.19
Blob 2: Location = (129.68, 592.70), Size = 23.62
Blob 3: Location = (97.25, 560.47), Size = 10.94
Blob 4: Location = (40.48, 565.32), Size = 25.10
Blob 5: Location = (213.22, 570.22), Size = 43.78
Blob 6: Location = (151.64, 530.82), Size = 10.18
Blob 7: Location = (1038.65, 525.23), Size = 11.51
Blob 8: Location = (914.54, 527.84), Size = 9.64
Blob 9: Location = (438.97, 537.70), Size = 28.67
Blob 10: Location = (5.02, 524.87), Size = 21.47
Blob 11: Location = (105.31, 516.31), Size = 11.63
Blob 12: Location = (224.97, 511.67), Size = 14.83
Blob 13: Location = (247.14, 499.51), Size = 10.52
Blob 14: Location = (438.46, 504.85), Size = 23.06
Blob 15: Location = (164.13, 484.46), Size = 9.26
Blob 16: Location = (226.79, 489.17), Size = 17.66
Blob 17: Location = (47.14, 482.68), Size = 14.76
Blob 18: Location = (576.66, 476.35), Size = 10.32
Blob 19: Location = (226.59, 469.71), Size = 12.11
Blob 20: Location = (352.62, 456.75), Size = 7.83
Blob 21: Location = (58.56, 456.61), Size = 14.31
Blob 22: Location = (319.56, 445.87), Size = 13.61
Blob 23: Location = (80.98, 438.69), Size = 10.52
Blob 24: Location = (107.18, 435.60), Size = 10.63
Blob 25: Location = (42.34, 435.85), Size = 12.66
Blob 26: Location = (579.31, 426.49), Size = 25.33
Blob 27: Location = (112.75, 399.18), Size = 9.12
Blob 28: Location = (523.39, 420.21), Size = 40.44
Blob 29: Location = (487.99, 381.03), Size = 17.27
Blob 30: Location = (214.52, 371.45), Size = 10.41
Blob 31: Location = (543.03, 346.43), Size = 19.44
Blob 32: Location = (622.79, 320.50), Size = 16.16
```

```
Blob 33: Location = (952.63, 313.51), Size = 11.22
Blob 34: Location = (411.94, 283.84), Size = 10.33
Blob 35: Location = (519.60, 283.96), Size = 19.75
Blob 36: Location = (91.00, 281.17), Size = 10.28
Blob 37: Location = (535.91, 278.06), Size = 15.01
Blob 38: Location = (352.33, 259.76), Size = 11.00
Blob 39: Location = (562.08, 246.22), Size = 14.67
Blob 40: Location = (326.93, 206.16), Size = 10.13
Blob 41: Location = (955.22, 187.46), Size = 39.12
Blob 42: Location = (48.17, 167.92), Size = 50.57
Blob 43: Location = (80.47, 133.02), Size = 49.41
Blob 44: Location = (918.51, 105.88), Size = 59.92
```

COMBINING TECHNIQUES: HIGHLIGHTING OBJECTS WITH CONTOURS

[25]:
```
#This line of code sets the kernel for the final image that utilizes multiple␣
 ↪techniques in performing image processing.
#In here, a 2x2 kernel is used for a finer operation of eroding and dilating.
final_kernel = np.ones((2,2), np.uint8)

#This utilizes the bitwise OR operation learned in Color Filtering under␣
 ↪Masking.
#This combines the otsu_thresh image and the combined_mask, which is a mix of␣
 ↪Automatic Thresholding and Color Filtering under Masking.
#The final mask intends to capture areas in the Otsu's Method Thresholding that␣
 ↪was not captured in the Combined Mask of Color Filtering.
final_mask = cv2.bitwise_or(otsu_thresh, combined_mask)

#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Final Mask Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(final_mask, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Final Mask')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
```

```
plt.show()
```



[26]:
```
#This line intends to dilate the mask and widen its white areas.
#This uses 2 iterations using the final kernel (a 2x2 kernel) for a finer␣
 ↪dilation and possibly refined white spaces.
mask_dilate = cv2.dilate(final_mask, final_kernel, iterations=2)

#This line of code performs an AND operation to the mask_dilate and the␣
 ↪adapt_thresh.
#This joins the common feature of the Adaptive Threshold and the dilated mask␣
 ↪above, storing it in the combined variable.
combined = cv2.bitwise_and(mask_dilate, adapt_thresh)

#This line intends to erode the combined mask and edges to broaden darker areas.
#This uses 1 iterations using the combined variable and the final_kernel (a 2x2␣
 ↪kernel) for a finer erosion and possibly refined thinned edges.
combined_erode = cv2.erode(combined, final_kernel, iterations=1)

#This performs an AND operation to an inversed 'combined_erode' and inversed␣
 ↪'adapt_thresh'.
#This captures the important areas of the two images, which preserves the edges␣
 ↪of the image to reveal shapes and features later.
for_final = cv2.bitwise_and(cv2.bitwise_not(combined_erode), cv2.
 ↪bitwise_not(adapt_thresh))
```

```
#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Combined Techniques Image.
#Cmap is still present in the 'imshow' parameter as this is a grayscale image,␣
 ↪without this the image would appear yellowish rather than grayscale.
plt.imshow(for_final, cmap='gray')
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Combined Otsu Method, Color Filtering (Mask and Operations), and␣
 ↪Adaptive Threshold')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



[27]:
```
#This line finds the contours of the white shapes in the 'for_final' image.
#The 'RETR_EXTERNAL' function gets the outermost contours while ignoring the␣
 ↪inner holes.
#The 'CHAIN_APPROX_SIMPLE' function reduces the points in the contour.
img_contours, _ = cv2.findContours(for_final, cv2.RETR_EXTERNAL, cv2.
 ↪CHAIN_APPROX_SIMPLE)

#This copies the original image 'rgb_img' to 'final_image'.
#This will be used later to display the image along with the rectangles to␣
 ↪emphasize recgonized features.
final_image = rgb_img.copy()

#This sets the minimum value for the contour.
#Only the objects above the value of 100 will be processed.
min_area = 100
#This sets the maximum value for the contour.
#Only the objects below the value of 10000 will be processed.
max_area = 10000

#This creates an empty list/array for storing object measurements or shape␣
 ↪properties.
#This will also be utilized for printing the stored information for later.
object_features = []

#This sets a loop in 'img_contours'.
#Basically, this goes through each contour (contour) found in the image.
for contour in img_contours:
    #This calculates the area of the contour.
    #Basically, it gets the number of pixels contained in the contour then␣
 ↪stores it in the variable 'area'.
    area = cv2.contourArea(contour)

    #Like any other if statements, this only continues processing if the area␣
 ↪is within the area range defined earlier.
    #Simply, the contour will only be recognized when it is between the minimum␣
 ↪and maximum area.
    if min_area <= area <= max_area:

        #This line gets the smallest rectangle that can surround the current␣
 ↪contour.
        #The variables on the left side gets the x and y values of the top-left␣
 ↪corner of the rectangle, then it height and width.
        x, y, w, h = cv2.boundingRect(contour)
        #The second parameter of the image provides the coordinates for the␣
 ↪top-left corner of the rectangle, while the thirs parameter provides the␣
 ↪bottom-right corner of the rectangle.
```

```
        #This adds a green rectangle on the final image, with a 2 pixel wide␣
↪line.
        cv2.rectangle(final_image, (x, y), (x + w, y + h), (0, 255, 0), 2)

        #Similar to the code above, this gets the smallest rectangle␣
↪surrounding the contour.
        #But on the other hand, using 'minAreaRect' allows for rotation to fit␣
↪the contour perfectly.
        #Rect will store a list of the information for the rotated bounding␣
↪rectangle.
        rect = cv2.minAreaRect(contour)
        #The stored 'rect' information will be utilized here to become vertices␣
↪of a rotated rectangle.
        #Box becomes a NumPy array to represent the vertices and coordinates of␣
↪the rotated rectangle.
        box = cv2.boxPoints(rect)
        #As the data type for the coordinates was stored as a float, this␣
↪converts its data type to an integer pixel.
        #This is needed as the function to draw the rectangle later expects␣
↪this data type.
        box = box.astype(np.intp)
        #This utilizes the vertices and coordinates of box.
        #This adds a blue rectangle around the shape, with still a 2 pixel wide␣
↪line.
        cv2.drawContours(final_image, [box], 0, (255, 0, 0), 2)

        #The function 'moments' helps calculate features such as the center of␣
↪mass of the object and its area (OpenCV, 2025).
        #Basically, this line computes a dictionary of raw moments for the␣
↪contour that will be necessary for determining the centroid for later.
        M = cv2.moments(contour)
        #The "m00" is the area of the contour, and checks if the area is not␣
↪equal to zero.
        #This helps avoid the zero division error, and executes the else␣
↪statement if the contour area is zero.
        if M["m00"] != 0:
            #The line of code below basically calculates for the x-coordinate␣
↪of the centroid.
            #Because the resulting number data type will be a float, it is␣
↪converted to an integer.
            cx = int(M["m10"] / M["m00"])
            #The line of code below basically calculates for the y-coordinate␣
↪of the centroid.
            #Because the resulting number data type will be a float, it is␣
↪converted to an integer.
            cy = int(M["m01"] / M["m00"])
```

```
            #This draws a circle on the final image that utilizes the computed␣
↪x and y coordinates 'cx' and 'cy' as the center coordinates with a radius of␣
↪4 pixels.
            #This blue circle will be filled instead of outlined because of the␣
↪value '-1' on its thickness parameter.
            cv2.circle(final_image, (cx, cy), 4, (0, 0, 255), -1)

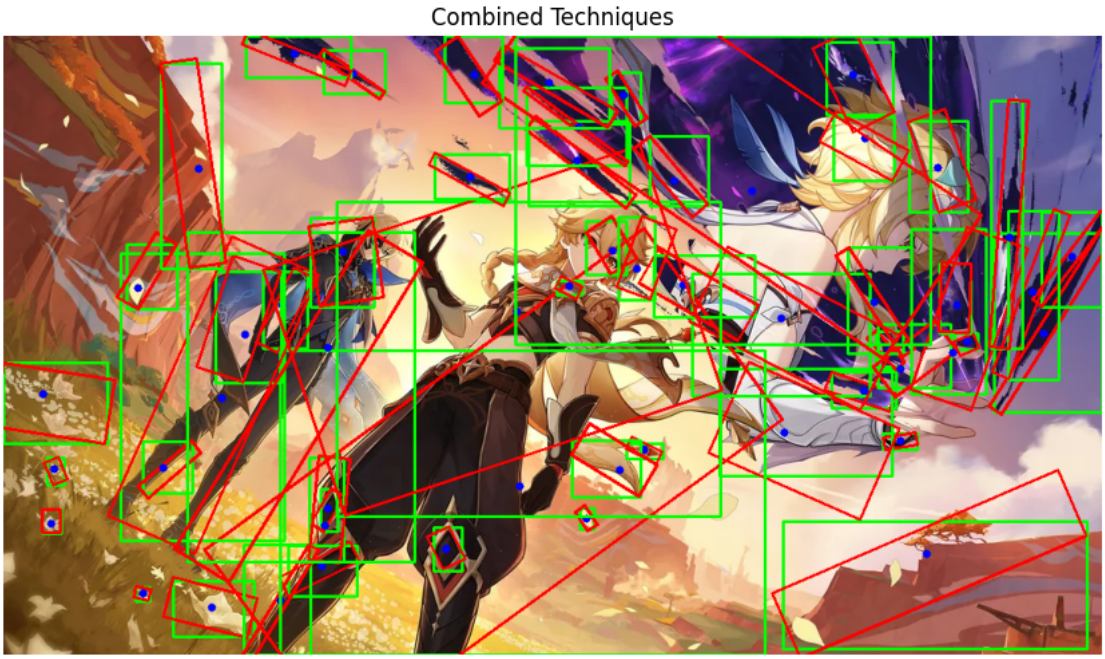
        #This is the else statement that will be executed once the contour area␣
↪is 0.
        #The x and y coordinates of the contour is 0 to avoid empty contours.
        else: cx, cy = 0, 0


        #This find the total length around the contour, or its perimeter.
        #The function uses the contour as its inputs and a flag that indicates␣
↪whether the curve is closed or not (OpenCV, 2025).
        perimeter = cv2.arcLength(contour, True)
        #The width of the rectangle is divided by its height, then stored in␣
↪the 'aspect_ratio' variable.
        #This gives the ratio of the contour, whether its shape is tall or wide.
        aspect_ratio = float(w) / h
        #Area is divided by the width and height multiplied together.
        #This tells how much the bounding rectangle is filled by the contour,␣
↪which value present is between 0 and 1.
        extent = area / (w * h)
        #The 'convexHull' function calculates the convex hull of the given␣
↪contour.
        #Simply, this line finds the tighest convex to bound the contour.
        hull = cv2.convexHull(contour)
        #Using the calculated hull from the line above, the function␣
↪'contourArea' will calculate the area of the polygon.
        #Basicallym this line measure the total area occupied by the convex␣
↪hull.
        hull_area = cv2.contourArea(hull)
        #This divides the area, that has been converted into float, to be␣
↪divided to the computed 'hull_area'.
        #The if statement is to avoid a zero division error.
        #This simply computes for the solidity of the object, describing how␣
↪filled in it is.
        solidity = float(area) / hull_area if hull_area != 0 else 0
        #Using the area of the contour multiplied by a constant, this is␣
↪divided by the NumPy pi, and squarerooted to get the diameter of the contour.
        #The calculation is then stored in 'equi_diameter' for printing later.
        equi_diameter = np.sqrt(4 * area / np.pi)
        #Utilizing the stored information on the rotated bounding rectangle, it␣
↪accesses the rotation angle of the rectangle.
```

```
        #This will then be stored for printing later to give context on the␣
 ↪object's orientation.
        angle = rect[2]

        #This line of code appends the information stored per object in the␣
 ↪object_features array list.
        #This takes the current information on the contour to be stored and␣
 ↪printed for later for extra information.
        object_features.append({
            "Area": area,
            "Perimeter": perimeter,
            "Aspect Ratio": aspect_ratio,
            "Extent": extent,
            "Solidity": solidity,
            "Equivalent Diameter": equi_diameter,
            "Orientation Angle": angle,
            "Centroid": (cx, cy)
        })
```

[28]:
```
#This function specifies the size of the image to be displayed.
#This creates a figure and uses figsize to specify the width and height in␣
 ↪inches.
plt.figure(figsize=(12, 6))
#Since pyplot mainly works with plot generation, normally axes will be seen.
#This function turns of the display of the axes, mainly to focus on the image␣
 ↪displayed.
plt.axis('off')
#This function specifies which stored image will be displayed, in this case, it␣
 ↪is the Final Image.
#The variables inputted will be rendered as a pseudocolor image (Matplotlib,␣
 ↪2025b).
plt.imshow(final_image)
#This function adds a title to the displayed image.
#In this title, it adds a description and context to its viewer.
plt.title('Combined Techniques')
#This is the last function to be executed after all the plotting commands.
#This basically shows the plotted image and the result of all our plotting␣
 ↪commands done earlier.
plt.show()
```



[29]:
```
#Below displays the measurements and calculations per identified feature in the picture.
#Each object_feature that has been in the array will be printed with its items.
for i, features in enumerate(object_features):
    print(f"\nObject {i+1}:")
    for key, value in features.items():
        print(f"  {key}: {value:.2f}" if isinstance(value, (float, int)) else f"  {key}: {value}")
```

```
Object 1:
  Area: 116.50
  Perimeter: 46.73
  Aspect Ratio: 1.00
  Extent: 0.59
  Solidity: 0.90
  Equivalent Diameter: 12.18
  Orientation Angle: 11.31
  Centroid: (138, 551)

Object 2:
  Area: 206.00
  Perimeter: 472.07
  Aspect Ratio: 1.45
  Extent: 0.05
  Solidity: 0.09
```

```
  Equivalent Diameter: 16.20
  Orientation Angle: 14.47
  Centroid: (206, 565)

Object 3:
  Area: 104.00
  Perimeter: 237.91
  Aspect Ratio: 1.36
  Extent: 0.03
  Solidity: 0.08
  Equivalent Diameter: 11.51
  Orientation Angle: 63.43
  Centroid: (315, 524)

Object 4:
  Area: 194.00
  Perimeter: 401.24
  Aspect Ratio: 0.61
  Extent: 0.03
  Solidity: 0.20
  Equivalent Diameter: 15.72
  Orientation Angle: 31.04
  Centroid: (274, 558)

Object 5:
  Area: 527.50
  Perimeter: 125.92
  Aspect Ratio: 0.65
  Extent: 0.44
  Solidity: 0.85
  Equivalent Diameter: 25.92
  Orientation Angle: 63.43
  Centroid: (438, 507)

Object 6:
  Area: 1209.50
  Perimeter: 1309.10
  Aspect Ratio: 2.39
  Extent: 0.03
  Solidity: 0.08
  Equivalent Diameter: 39.24
  Orientation Angle: 66.72
  Centroid: (914, 512)

Object 7:
  Area: 248.00
  Perimeter: 72.28
  Aspect Ratio: 0.79
```

```
  Extent: 0.54
  Solidity: 0.83
  Equivalent Diameter: 17.77
  Orientation Angle: 0.00
  Centroid: (47, 482)

Object 8:
  Area: 182.00
  Perimeter: 55.46
  Aspect Ratio: 0.67
  Extent: 0.62
  Solidity: 0.96
  Equivalent Diameter: 15.22
  Orientation Angle: 56.31
  Centroid: (577, 477)

Object 9:
  Area: 315.00
  Perimeter: 96.91
  Aspect Ratio: 0.40
  Extent: 0.44
  Solidity: 0.86
  Equivalent Diameter: 20.03
  Orientation Angle: 30.96
  Centroid: (321, 467)

Object 10:
  Area: 588.00
  Perimeter: 376.82
  Aspect Ratio: 0.33
  Extent: 0.17
  Solidity: 0.28
  Equivalent Diameter: 27.36
  Orientation Angle: 5.27
  Centroid: (318, 484)

Object 11:
  Area: 135.50
  Perimeter: 96.33
  Aspect Ratio: 0.63
  Extent: 0.30
  Solidity: 0.52
  Equivalent Diameter: 13.13
  Orientation Angle: 68.20
  Centroid: (50, 428)

Object 12:
  Area: 169.50
```

```
  Perimeter: 97.15
  Aspect Ratio: 2.40
  Extent: 0.31
  Solidity: 0.57
  Equivalent Diameter: 14.69
  Orientation Angle: 26.57
  Centroid: (634, 409)

Object 13:
  Area: 153.50
  Perimeter: 249.06
  Aspect Ratio: 0.96
  Extent: 0.06
  Solidity: 0.23
  Equivalent Diameter: 13.98
  Orientation Angle: 49.90
  Centroid: (158, 427)

Object 14:
  Area: 110.50
  Perimeter: 380.37
  Aspect Ratio: 1.21
  Extent: 0.03
  Solidity: 0.06
  Equivalent Diameter: 11.86
  Orientation Angle: 31.55
  Centroid: (610, 429)

Object 15:
  Area: 124.00
  Perimeter: 90.57
  Aspect Ratio: 2.12
  Extent: 0.23
  Solidity: 0.48
  Equivalent Diameter: 12.57
  Orientation Angle: 80.22
  Centroid: (888, 400)

Object 16:
  Area: 124.50
  Perimeter: 317.66
  Aspect Ratio: 1.78
  Extent: 0.05
  Solidity: 0.12
  Equivalent Diameter: 12.59
  Orientation Angle: 18.71
  Centroid: (852, 351)
```

```
Object 17:
  Area: 682.00
  Perimeter: 1679.53
  Aspect Ratio: 1.61
  Extent: 0.04
  Solidity: 0.06
  Equivalent Diameter: 29.47
  Orientation Angle: 23.75
  Centroid: (773, 392)

Object 18:
  Area: 1586.50
  Perimeter: 1033.16
  Aspect Ratio: 1.29
  Extent: 0.19
  Solidity: 0.31
  Equivalent Diameter: 44.94
  Orientation Angle: 8.82
  Centroid: (39, 354)

Object 19:
  Area: 5792.00
  Perimeter: 5627.20
  Aspect Ratio: 1.59
  Extent: 0.04
  Solidity: 0.07
  Equivalent Diameter: 85.88
  Orientation Angle: 53.94
  Centroid: (511, 445)

Object 20:
  Area: 152.00
  Perimeter: 64.63
  Aspect Ratio: 1.44
  Extent: 0.41
  Solidity: 0.79
  Equivalent Diameter: 13.91
  Orientation Angle: 66.04
  Centroid: (954, 302)

Object 21:
  Area: 203.00
  Perimeter: 132.85
  Aspect Ratio: 0.77
  Extent: 0.21
  Solidity: 0.67
  Equivalent Diameter: 16.08
  Orientation Angle: 47.49
```

```
  Centroid: (877, 311)

Object 22:
  Area: 488.00
  Perimeter: 336.53
  Aspect Ratio: 1.14
  Extent: 0.11
  Solidity: 0.23
  Equivalent Diameter: 24.93
  Orientation Angle: 53.13
  Centroid: (888, 329)

Object 23:
  Area: 170.50
  Perimeter: 89.84
  Aspect Ratio: 1.79
  Extent: 0.49
  Solidity: 0.73
  Equivalent Diameter: 14.73
  Orientation Angle: 26.57
  Centroid: (560, 247)

Object 24:
  Area: 1770.00
  Perimeter: 1552.13
  Aspect Ratio: 1.45
  Extent: 0.08
  Solidity: 0.16
  Equivalent Diameter: 47.47
  Orientation Angle: 27.41
  Centroid: (770, 279)

Object 25:
  Area: 244.00
  Perimeter: 492.50
  Aspect Ratio: 0.58
  Extent: 0.07
  Solidity: 0.17
  Equivalent Diameter: 17.63
  Orientation Angle: 26.57
  Centroid: (276, 266)

Object 26:
  Area: 224.50
  Perimeter: 616.54
  Aspect Ratio: 0.59
  Extent: 0.03
  Solidity: 0.08
```

```
  Equivalent Diameter: 16.91
  Orientation Angle: 18.28
  Centroid: (239, 295)

Object 27:
  Area: 115.50
  Perimeter: 243.72
  Aspect Ratio: 0.45
  Extent: 0.05
  Solidity: 0.08
  Equivalent Diameter: 12.13
  Orientation Angle: 2.16
  Centroid: (944, 266)

Object 28:
  Area: 1645.50
  Perimeter: 451.00
  Aspect Ratio: 1.64
  Extent: 0.27
  Solidity: 0.55
  Equivalent Diameter: 45.77
  Orientation Angle: 30.02
  Centroid: (673, 246)

Object 29:
  Area: 1161.50
  Perimeter: 1516.29
  Aspect Ratio: 0.57
  Extent: 0.03
  Solidity: 0.05
  Equivalent Diameter: 38.46
  Orientation Angle: 24.04
  Centroid: (216, 358)

Object 30:
  Area: 195.50
  Perimeter: 298.29
  Aspect Ratio: 0.60
  Extent: 0.03
  Solidity: 0.14
  Equivalent Diameter: 15.78
  Orientation Angle: 56.69
  Centroid: (862, 263)

Object 31:
  Area: 390.00
  Perimeter: 213.76
  Aspect Ratio: 0.78
```

```
  Extent: 0.12
  Solidity: 0.39
  Equivalent Diameter: 22.28
  Orientation Angle: 30.83
  Centroid: (133, 249)

Object 32:
  Area: 239.50
  Perimeter: 483.91
  Aspect Ratio: 0.60
  Extent: 0.01
  Solidity: 0.13
  Equivalent Diameter: 17.46
  Orientation Angle: 32.41
  Centroid: (1030, 294)

Object 33:
  Area: 2552.00
  Perimeter: 1547.36
  Aspect Ratio: 0.69
  Extent: 0.03
  Solidity: 0.16
  Equivalent Diameter: 57.00
  Orientation Angle: 31.33
  Centroid: (321, 308)

Object 34:
  Area: 550.50
  Perimeter: 869.60
  Aspect Ratio: 0.45
  Extent: 0.05
  Solidity: 0.08
  Equivalent Diameter: 26.47
  Orientation Angle: 19.33
  Centroid: (940, 313)

Object 35:
  Area: 188.00
  Perimeter: 358.33
  Aspect Ratio: 0.75
  Extent: 0.09
  Solidity: 0.16
  Equivalent Diameter: 15.47
  Orientation Angle: 59.53
  Centroid: (603, 212)

Object 36:
  Area: 283.50
```

```
  Perimeter: 753.28
  Aspect Ratio: 0.72
  Extent: 0.05
  Solidity: 0.09
  Equivalent Diameter: 19.00
  Orientation Angle: 82.12
  Centroid: (336, 213)

Object 37:
  Area: 240.00
  Perimeter: 346.33
  Aspect Ratio: 0.44
  Extent: 0.08
  Solidity: 0.15
  Equivalent Diameter: 17.48
  Orientation Angle: 20.56
  Centroid: (627, 230)

Object 38:
  Area: 109.50
  Perimeter: 261.32
  Aspect Ratio: 0.64
  Extent: 0.02
  Solidity: 0.15
  Equivalent Diameter: 11.81
  Orientation Angle: 37.45
  Centroid: (1058, 218)

Object 39:
  Area: 180.00
  Perimeter: 444.19
  Aspect Ratio: 0.30
  Extent: 0.02
  Solidity: 0.10
  Equivalent Diameter: 15.14
  Orientation Angle: 20.71
  Centroid: (1019, 253)

Object 40:
  Area: 9016.00
  Perimeter: 6174.94
  Aspect Ratio: 1.22
  Extent: 0.08
  Solidity: 0.13
  Equivalent Diameter: 107.14
  Orientation Angle: 71.66
  Centroid: (508, 272)
```

```
Object 41:
  Area: 273.00
  Perimeter: 354.62
  Aspect Ratio: 1.68
  Extent: 0.08
  Solidity: 0.34
  Equivalent Diameter: 18.64
  Orientation Angle: 28.44
  Centroid: (462, 139)

Object 42:
  Area: 406.00
  Perimeter: 227.08
  Aspect Ratio: 0.84
  Extent: 0.10
  Solidity: 0.38
  Equivalent Diameter: 22.74
  Orientation Angle: 49.61
  Centroid: (664, 143)

Object 43:
  Area: 272.00
  Perimeter: 903.72
  Aspect Ratio: 0.64
  Extent: 0.05
  Solidity: 0.10
  Equivalent Diameter: 18.61
  Orientation Angle: 71.35
  Centroid: (925, 130)

Object 44:
  Area: 1098.00
  Perimeter: 381.04
  Aspect Ratio: 1.19
  Extent: 0.13
  Solidity: 0.68
  Equivalent Diameter: 37.39
  Orientation Angle: 38.45
  Centroid: (568, 123)

Object 45:
  Area: 131.00
  Perimeter: 365.16
  Aspect Ratio: 0.95
  Extent: 0.03
  Solidity: 0.07
  Equivalent Diameter: 12.91
  Orientation Angle: 28.52
```

```
  Centroid: (853, 101)

Object 46:
  Area: 587.00
  Perimeter: 934.24
  Aspect Ratio: 0.13
  Extent: 0.08
  Solidity: 0.19
  Equivalent Diameter: 27.34
  Orientation Angle: 4.47
  Centroid: (996, 199)

Object 47:
  Area: 171.00
  Perimeter: 296.13
  Aspect Ratio: 1.29
  Extent: 0.02
  Solidity: 0.35
  Equivalent Diameter: 14.76
  Orientation Angle: 36.63
  Centroid: (569, 90)

Object 48:
  Area: 470.00
  Perimeter: 124.02
  Aspect Ratio: 0.57
  Extent: 0.32
  Solidity: 0.91
  Equivalent Diameter: 24.46
  Orientation Angle: 59.04
  Centroid: (614, 58)

Object 49:
  Area: 479.00
  Perimeter: 731.52
  Aspect Ratio: 0.30
  Extent: 0.04
  Solidity: 0.09
  Equivalent Diameter: 24.70
  Orientation Angle: 81.82
  Centroid: (193, 131)

Object 50:
  Area: 114.50
  Perimeter: 178.21
  Aspect Ratio: 1.42
  Extent: 0.04
  Solidity: 0.33
```

```
  Equivalent Diameter: 12.07
  Orientation Angle: 35.26
  Centroid: (346, 37)

Object 51:
  Area: 182.00
  Perimeter: 327.59
  Aspect Ratio: 1.38
  Extent: 0.02
  Solidity: 0.14
  Equivalent Diameter: 15.22
  Orientation Angle: 30.11
  Centroid: (540, 46)

Object 52:
  Area: 306.50
  Perimeter: 644.83
  Aspect Ratio: 0.92
  Extent: 0.06
  Solidity: 0.12
  Equivalent Diameter: 19.75
  Orientation Angle: 64.54
  Centroid: (841, 38)

Object 53:
  Area: 4124.50
  Perimeter: 5755.50
  Aspect Ratio: 1.35
  Extent: 0.03
  Solidity: 0.07
  Equivalent Diameter: 72.47
  Orientation Angle: 34.49
  Centroid: (741, 153)

Object 54:
  Area: 191.50
  Perimeter: 351.00
  Aspect Ratio: 0.86
  Extent: 0.05
  Solidity: 0.15
  Equivalent Diameter: 15.61
  Orientation Angle: 55.86
  Centroid: (466, 38)

Object 55:
  Area: 167.50
  Perimeter: 334.09
  Aspect Ratio: 2.54
```

```
Extent: 0.04
Solidity: 0.14
Equivalent Diameter: 14.60
Orientation Angle: 22.04
Centroid: (288, 17)
```

BIBLIOGRAPHY

Buhl, N. (2023, September 12). Image Thresholding in Image Processing. Encord.

GeeksforGeeks. (2016, December 13). RealTime Edge Detection using OpenCV in Python | Canny edge detection method. GeeksforGeeks.

GeeksforGeeks. (2017a, May 7). Erosion and Dilation of images using OpenCV in python. GeeksforGeeks.

GeeksforGeeks. (2017b, July 22). numpy.ones() in Python. GeeksforGeeks.

GeeksforGeeks. (2019a, April 17). Python | Image blurring using OpenCV. GeeksforGeeks.

GeeksforGeeks. (2019b, May 27). Python | Thresholding techniques using OpenCV | Set3 (Otsu Thresholding). GeeksforGeeks.

GeeksforGeeks. (2019c, July 9). Filter Color with OpenCV. GeeksforGeeks.

GeeksforGeeks. (2019d, September 2). Python | Bilateral Filtering. GeeksforGeeks.

Jarvis, D. (2024, November 30). [CV2] HSV vs RGB: Understanding and Leveraging HSV for Image Processing. DEV Community.

Kulhary, R. (2019, September 23). OpenCV - Overview. GeeksforGeeks.

Matplotlib. (2025a). Matplotlib 3.5.3 documentation. Matplotlib.org.

Matplotlib. (2025b). matplotlib.pyplot.imshow — Matplotlib 3.5.2 documentation. Matplotlib.org.

Matplotlib. (2025c). matplotlib.pyplot.tight_layout — Matplotlib 3.5.1 documentation. Matplotlib.org.

Murzova, A., & Seth, S. (2020, August 5). Otsu's Thresholding Technique | Learn OpenCV. LearnOpenCV.

OpenCV. (2025). OpenCV: Contour Features. OpenCV.

Python. (2010). sys — System-specific parameters and functions — Python 3.7.3 documentation. Python.

RoboFlow. (2025, February 27). OpenCV Color Spaces and Conversion: An Introduction. Roboflow Blog.

TutorialsPoint. (2025). OpenCV - Adaptive Threshold. TutorialsPoint.

W3Schools. (2022). Introduction to NumPy. W3Schools.